Assignment 2 Sorting Techniques part 2

Rodina Mohamed 9302

Rodina Ahmed 9712

Sara Yasser 9334

Output:

```
Hybrid Merge and Selection Sort
Before: [23, 25, 4, 7, 11, 48, 3, 10, 43]
After: [3, 4, 7, 10, 11, 23, 25, 43, 48]
Array size : 5000
O(n^2)
bubble sort time: 2106.966018676758 ms
selection sort time: 749.9887943267822 ms
insertion sort time: 781.6941738128662 ms
O(n log n)
Heap sort time: 19.100427627563477 ms
Quick sort time: 13.267040252685547 ms
Merge sort time: 16.5860652923584 ms

Array size : 10000
O(n^2)
bubble sort time: 7230.529069900513 ms
selection sort time: 2583.65535736084 ms
insertion sort time: 2964.9014472961426 ms
O(n log n)
Heap sort time: 38.54632377624512 ms
Quick sort time: 17.84515380859375 ms
Merge sort time: 26.439189910888672 ms

Array size : 25000
O(n^2)
bubble sort time: 44471.64583206177 ms
selection sort time: 18933.90393257141 ms
insertion sort time: 20275.765419006348 ms
```

rting-Alogrithms > Sorting_Algorithms.py 236:1

```
Heap sort time: 19.100427627563477 ms
Quick sort time: 13.267040252685547 ms
Merge sort time: 16.5860652923584 ms

Array size : 10000
O(n^2)
bubble sort time: 7230.529069900513 ms
selection sort time: 2583.65535736084 ms
insertion sort time: 2964.9014472961426 ms
O(n log n)
Heap sort time: 38.54632377624512 ms
Quick sort time: 17.84515380859375 ms
Merge sort time: 26.439189910888672 ms

Array size : 25000
O(n^2)
bubble sort time: 44471.64583206177 ms
selection sort time: 18933.90393257141 ms
insertion sort time: 20275.765419006348 ms
O(n log n)
Heap sort time: 152.3454189300537 ms
Quick sort time: 55.57370185852051 ms
Merge sort time: 71.12431526184082 ms

Kth smallest element = 3

Process finished with exit code 0
```

Running times of sorting algorithms for different array sizes:

Bubble Sort, Selection Sort, and Insertion Sort are $O(n^2)$ algorithms. Bubble Sort is the slowest for all array sizes due to the high number of comparisons and swaps. Insertion Sort performs slightly better than Selection Sort on larger arrays, especially if the array is partially sorted.

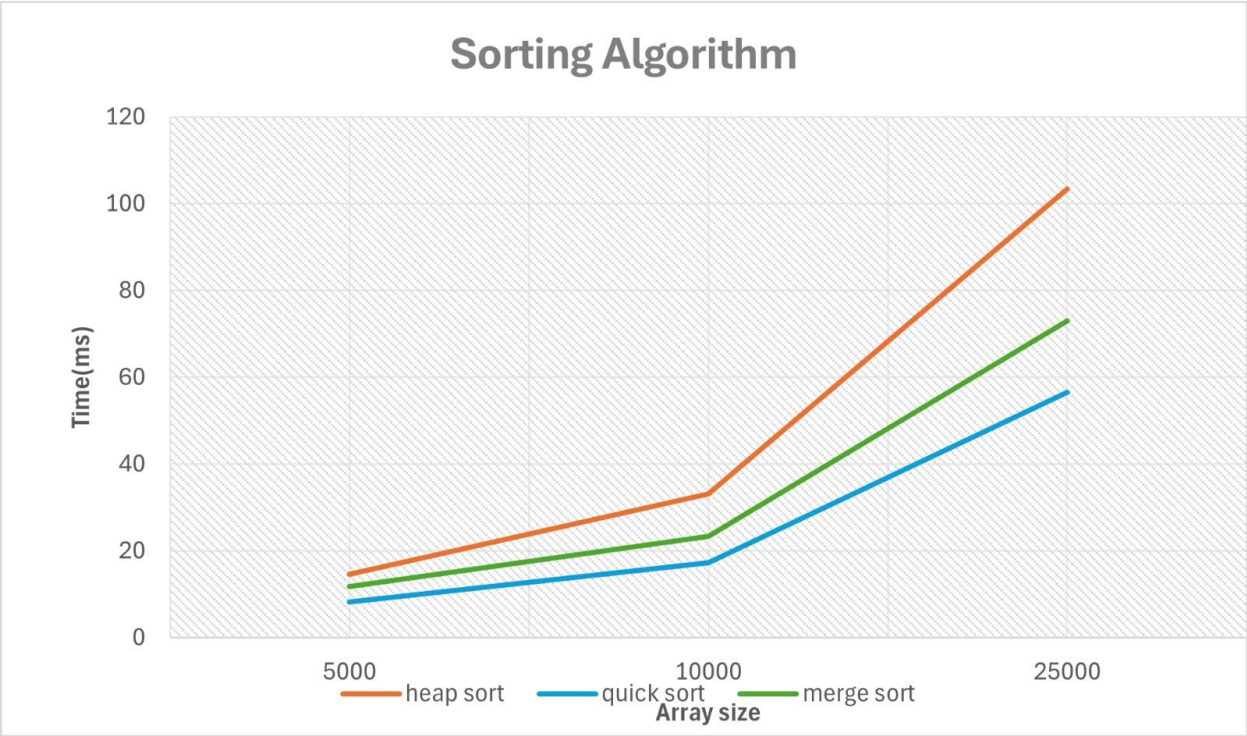
Heap Sort, Quick Sort, and Merge Sort are O(n log n) algorithms and are much faster even for large arrays. Heap Sort has slightly higher times than Quick Sort in these experiments, while Merge Sort performs consistently well. Hybrid Merge-Selection Sort combines Merge Sort with Selection Sort for small subarrays to improve efficiency.

Measured running times (ms):

- Array size 5000: Bubble = 933, Selection = 351, Insertion = 352, Heap = 9, Quick = 5, Merge = 8
- Array size 10000: Bubble = 3179, Selection = 1392, Insertion = 1434, Heap = 24, Quick = 12, Merge = 19
- Array size 25000: Bubble = 20304, Selection = 8698, Insertion = 8955, Heap = 59, Quick = 30, Merge = 45

K-th smallest element in the test array [7, 10, 4, 3, 20, 15] (k=1) is 3.

Graph:



Quick Sort is the fastest algorithm in these experiments due to its efficient divide-and-conquer strategy with random pivot selection. Heap Sort also performs very well, slightly slower than Quick Sort, because of heap construction and maintenance. Merge Sort shows consistent performance across all array sizes due to its stable O(n log n) complexity and systematic merging process. Time is measured in milliseconds (ms).